

The Hub

Build Plan — Technical Reference

Phases, steps, and confirmed technology stack for constructing the full-scope marketplace and discovery platform. Companion to the authoritative Project Scope; this document sequences and architects the build.

STATUS	Approved for planning	VERSION	1.0	DATE	June 23, 2026
---------------	-----------------------	----------------	-----	-------------	---------------

Authors: Michel + Claude

Build model: Solo builder + Claude · full scope, no feature cuts · token-frugal process

Source of truth (scope): The Hub - Project Scope.md (19 sections)

Source of truth (build): docs/superpowers/specs/2026-06-23-the-hub-build-plan-design.md

Launch market: Lebanon (architecture not hardcoded to one geography)

1. Purpose & Constraints

How a solo builder plus Claude constructs the full-scope product without sacrificing features or quality, while keeping Claude token cost low during the build.

1. Full scope. Every feature in the Project Scope ships. Nothing is cut.

2. No quality sacrifice. Architecture choices must not lower the quality ceiling; security, correctness, and UX stay first-class.

3. Token-frugal process. Frugality applies to *how we work*, never to *what gets built*. It refers to Claude usage during the build, not the product's runtime cost.

How token-frugality is achieved structurally

- **Vertical slices, one at a time.** Each feature is built end-to-end (schema → service → API → UI → test), committed, then the next begins. Small focused files keep Claude's working context small and edits reliable.
- **Living architecture map.** A maintained `CLAUDE.md` plus the memory store let each session boot cheaply without re-reading the full scope doc.
- **No subagent fan-out by default.** Work proceeds in the main thread unless a task genuinely needs isolation.
- **Caveman communication mode** active for chat (substance unchanged, ~75% fewer tokens).
- **Decisions batched;** context reused within a session.

2. Architecture Decision (Approved)

The Project Scope suggests a polyglot, multi-service stack. For a solo + Claude build, a leaner unified stack delivers identical user-facing features at higher quality (fewer integration seams = fewer bugs) and lower token cost. Four swaps were reviewed and all four approved.

#	Scope suggested	Approved choice (V1)	Why equal-or-better here	Upgrade path
A	NestJS API + separate Next.js web	One Next.js full-stack app (App Router + route handlers / server actions) over a clean <code>services/</code> layer	One codebase / deploy / language; shared TS types catch mismatches at compile time; RBAC enforced server-side per scope §2/§12.2. Two-repo split only pays off at team size.	Lift <code>services/</code> into a standalone NestJS service when a team or independent API scaling is needed.
B	Elasticsearch / Algolia	Postgres full-text search + PostGIS	Every \$4.2 search/filter/geo/sort feature present; no index-sync bug surface; fast at Lebanon-MVP scale; multilingual FTS (ar/en/fr).	Add Elasticsearch as a read index fed from Postgres when catalog/traffic demand it; interface unchanged.
C	Python FastAPI recsys from start	TS rule-based recsys now; Python ML service at Phase 4	Matches scope §8's own cold-start rollout — ML has no data to train on at launch. Rule-based (interests → category tags) is trivial in TS.	Build the Python FastAPI hybrid service in Phase 4, A/B tested vs the rule-based baseline.
D	Web + mobile in parallel	Web first, Expo mobile after core proven	Matches scope §16 phasing; mobile reuses settled, tested TS logic instead of chasing a moving core.	Ship iOS + Android via Expo in Phase 5 with full deep-linking; nothing cut.

Everything else stays fully in: RBAC, phone OTP verification, TOTP MFA, claim/registration/approval lifecycle, reviews + votes + reviewer score, content moderation, notifications, boosts + agency discounts, payment-provider abstraction (2Checkout/Verifone), i18n ar/en/fr + RTL, the full §12 security checklist, and analytics/interaction logging from day one.

Confirmed Stack (V1)

Layer	Choice
Language	TypeScript end-to-end (Python only at Phase 4 ML)
App	Next.js 16 (App Router) + Tailwind CSS, modular <code>services/</code> layer
Database	PostgreSQL 16 + PostGIS, accessed via Prisma 7 (<code>@prisma/adapters-pg</code> driver adapter; engine-less client)
Search	Postgres full-text + PostGIS (no external engine at launch)

Auth	Auth.js / NextAuth v5 — OAuth social for End Users; email/password for Agency & Individual Location; server-side RBAC (<code>hasRole</code> / <code>assertRole</code>)
File storage	AWS S3 or Cloudflare R2, signed/expiring URLs
Push / Email	Firebase Cloud Messaging; transactional email (SendGrid / Mailgun)
SMS OTP	Provider TBD — Lebanon delivery confirmed before launch (scope risk \$18)
Payments	Provider abstraction; 2Checkout / Verifone as first implementation
i18n	next-intl v4, <code>[locale]</code> segment (en/ar/fr), RTL via direction helper; locale routing in <code>src/proxy.ts</code>
ML (Phase 4)	Python FastAPI microservice (hybrid content + collaborative filtering)
Mobile (Phase 5)	Expo / React Native, sharing TS logic + API client
Ops	Error monitoring (Sentry, guarded), CI, secrets manager, WAF/DDoS at hardening

3. Phase Roadmap

The scope's three-phase plan (§16) is refined into six finer, solo-buildable slices. No features are removed — open risks (§18) are tracked as gates inside the phase that depends on them.

PHASE 0

DONE

Foundations

Project skeleton and cross-cutting concerns wired before any feature work.

- Monorepo + Next.js app skeleton; PostgreSQL + PostGIS provisioned; Prisma baseline.
- Auth.js shell; **server-side RBAC middleware**.
- i18n (ar/en/fr) + RTL wired from the start.
- Secrets manager; CI pipeline; error monitoring (Sentry, guarded).
- **Interaction / analytics logging live from day one** (training data for later ML).

Scope refs: §2, §9, §11, §12, §14

PHASE 1

DONE

Catalog + Discovery

The searchable place catalog and the discovery experience over it.

- Core data model (§6); category taxonomy — 16 parents, ~80 subcategories (§3).
- Admin bulk-seed / import for initial database seeding.
- Location profile page (§4.3).
- Search & discovery via Postgres FTS + PostGIS — keyword, category/subcategory, distance radius, price, rating threshold, open-now, sorting, boosted-listing surfacing (§4.2).
- Favorites + custom lists + share links + deep-link foundation (§4.5).

Scope refs: §3, §4.2, §4.3, §4.5, §5.1, §6

PHASE 2

TO DO

Accounts + Trust

Identity, verification, the listing lifecycle, and the review/trust system.

- Full auth (OAuth + email per role); phone OTP verification (§5).
- TOTP MFA — mandatory for Admin / Super Admin (§12.1).
- Listing lifecycle — claim, self-registration, Admin approval gate (§5).
- Reviews + ratings + photos + votes + reviewer score (§4.4).
- Content moderation queue + fake-review / conflict-of-interest heuristics (§13).
- Notifications across push / email / in-app (§4.6).
- Onboarding questionnaire (§4.1) + rule-based recommendations (§8 phase 1).

Scope refs: §4.1, §4.4, §4.6, §4.7, §5, §8(P1), §12.1, §13

PHASE 3

TO DO

Monetization

The pay-per-boost revenue flow and the consoles that drive it.

- Boost / Campaign model; boost-eligibility rating floor (§7).
- Agency discount (flat or volume-tiered, stored as a rate on the Agency record).
- Checkout through the payment abstraction (2Checkout/Verifone) with no-refund disclosure at checkout (§7, §10).
- Agency console (§4.8); Admin console completion (§4.7).
- Analytics dashboard surfacing the day-one logged metrics (§14).

Scope refs: §4.7, §4.8, §7, §10, §14

PHASE 4

TO DO

Intelligence

Machine-learning recommendations and the conversational layer on top of them.

- Python FastAPI recsys microservice — hybrid content + collaborative filtering, evaluation harness, A/B test vs rule-based baseline (§8 phase 2+).
- LLM recommendation chatbot as a conversational layer that calls the recsys engine via tool-calling — never reasons over the dataset itself — with prompt-injection guards and abuse flagging into the moderation queue (§4.9).

Scope refs: §4.9, §8(P2)

PHASE 5

TO DO

Mobile + Hardening

Native clients and the pre-launch security, legal, and accessibility gates.

- Expo iOS + Android app sharing TS logic.
- Full iOS Universal Links / Android App Links deep-linking with app-store fallback (§9, §11).
- Security hardening — WAF + DDoS mitigation, third-party pentest, tested backup/restore, written incident-response plan (§12.6).
- Legal — ToS + Privacy Policy with lawyer sign-off (§12.7).
- Accessibility + RTL / localization QA pass (§15).

Scope refs: §9, §11, §12.6, §12.7, §15

4. Risk Gates (from Scope §18)

Each open risk is tracked inside the phase that first depends on it — none are surprises. Items with external lead time (payments, SMS) should start early, in parallel with build work.

Risk	Gated in
2Checkout / Verifone merchant approval for a Lebanon entity	Phase 3 — start application early; it blocks revenue
SMS / OTP provider reliable in Lebanon	Phase 2 — blocks listing go-live
Initial data-seeding source (Google-Maps-breadth dataset)	Phase 1
Claim ownership-verification method	Phase 2
Agency discount model (flat vs volume-tiered)	Phase 3
ML cold-start (needs accumulated interaction data)	Phase 4 — logging starts Phase 0
Local legal compliance (Law 81/2018)	Phase 5

5. Working Agreement

- One vertical slice at a time; **commit on green**.
- Security controls (§12) are built *with* each feature, not bolted on later.
- `CLAUDE.md` is kept current as the architecture map so sessions start cheap.
- Each phase produces its own implementation plan when it begins; this doc is the umbrella.
- **TDD**: failing test first, then minimal implementation (Vitest); DB-backed tests run in the node environment.
- Server-side RBAC on every protected action — never UI-only. Push to origin after every completed task.

Current state. Phase 0 foundations complete (CI on Node 22 / pnpm, guarded Sentry init, architecture map committed). Phase 1 Catalog + Discovery services in place: `taxonomy`, `locations`, `search`, `favorites`, `import-locations`. Deferred hooks noted in code: secondary-category search, open-now (needs Phase 2 hours schema), boosted surfacing (Phase 3), search GIN/GiST indexes (hardening).